Manual técnico — Sistema de Costos

Versión 1.0.0 · Costeo estándar (materiales, mano de obra directa, CIF y variaciones)

1. Arquitectura

```
Navegador (HTML + CSS + JavaScript)
        │  peticiones HTTP/JSON  (API REST, token en cabecera Authorization)
        ▼
Backend Python 3 (Flask)  ──►  Motor de cálculo (calculos.py) · Reportes Excel/PDF (reportes.py)
        │  sqlite3 (incluido en Python)
        ▼
SQLite: archivo database/sistema_costos.db   (opcional: MySQL 8 vía PyMySQL)
```

- **Ejecución 100 % local**: el servidor escucha en `127.0.0.1:5000`; no usa internet ni librerías por CDN (los gráficos son SVG propios).
- **Frontend**: HTML, CSS y JavaScript sin frameworks ni compilación. Lo sirve el mismo backend.
- **Backend**: Python con Flask; en producción se ejecuta con *waitress*.
- **Base de datos**: SQLite, un único archivo local (`database/sistema_costos.db`). No requiere instalar ni configurar ningún servidor. Se usa modo WAL e integridad referencial activada. **Opcional**: MySQL 8 cambiando `motor = mysql` (ver sección 4.3).
- **Comunicación**: exclusivamente por API REST con JSON.

2. Estructura de carpetas

La carpeta principal contiene **solo carpetas**; cada archivo está en la carpeta de su función.

```
SistemaCostos/
├── backend/                 API y lógica de negocio (Python + Flask)
│   ├── requirements.txt     Dependencias de Python
│   ├── app.py               Punto de entrada: inicia la API y sirve el frontend
│   ├── configuracion.py     Lectura de config/config.ini
│   ├── bd.py                Conexión SQLite o MySQL y creación del esquema
│   ├── seguridad.py         Sesiones, roles, permisos, contraseñas, bitácora
│   ├── validaciones.py      Reglas de validación y mensajes de error
│   ├── recursos.py          Definición declarativa de catálogos (campos y reglas)
│   ├── calculos.py          Motor de costeo estándar y variaciones
│   ├── reportes.py          Construcción de reportes y exportación Excel/PDF
│   └── rutas/               Endpoints de la API
│       ├── usuarios.py      Autenticación, registro, usuarios, permisos, bitácora
│       ├── catalogos.py     CRUD genérico de catálogos
│       ├── produccion.py    Órdenes, hoja de costos, variaciones
│       ├── tablero.py       Dashboard (costo unitario, consumos, CIF aplicado y real) y reportes
│       └── comun.py         Utilidades (empresa activa, periodos)
├── frontend/                Interfaz (HTML, CSS, JavaScript)
│   ├── index.html           Inicio de sesión y creación de cuenta
│   ├── app.html             Aplicación
│   ├── css/estilos.css      Estilos
│   ├── img/icono.svg        Ícono del sistema
│   └── js/                  Lógica de pantallas: api.js, ui.js, app.js... y modulos/
├── database/
│   ├── schema.sql           Esquema completo (se adapta automáticamente a SQLite)
│   └── sistema_costos.db    Base de datos SQLite (incluye los datos de la empresa Deligaby)
├── config/config.ini        Parámetros del sistema
├── datos/                   Archivos de datos para importar (Costeo_Deligaby.xlsx)
├── herramientas/
│   ├── IMPORTAR_EXCEL.bat   Doble clic: carga la plantilla de costeo en Excel (carpeta datos/)
│   └── importar_excel.py    Importador (usa la API del sistema)
├── instalador/              Instalación, arranque y acceso directo
│   ├── INSTALAR_TODO.bat    Doble clic: instala todo en Windows (Python, MySQL, base de datos, acceso
directo)
│   ├── INICIAR_SISTEMA.bat  Abre el sistema (lo usa el acceso directo del escritorio)
│   ├── CREAR_ACCESO_DIRECTO.bat  Crea "Sistema de Costos" en el escritorio y el menú Inicio
│   ├── instalar_todo.ps1    Lógica de INSTALAR_TODO.bat
│   ├── iniciar.bat          Arranque de la versión instalada (Windows)
│   ├── instalar.sh          Instalación en macOS/Linux
│   ├── iniciar.sh           Arranque en macOS/Linux
│   ├── desinstalar.bat      Desinstalación
│   ├── construir_exe.bat    Genera el ejecutable y el instalador
│   ├── SistemaCostos.spec   Configuración de PyInstaller (EXE)
│   ├── SistemaCostos.iss    Configuración de Inno Setup (instalador .exe)
│   └── icono.ico            Ícono del acceso directo y del ejecutable
├── docs/                    README, manual técnico y manual de usuario (Markdown y PDF)
├── tests/test_sistema.py    Pruebas automáticas
├── logs/                    Registro de errores (sistema.log)
├── reportes_generados/      Copia de los reportes exportados
├── mysql/                   MySQL 8.4 propio del sistema (lo crea el instalador; no viene en el zip)
└── .github/workflows/       Construcción automática del instalador en GitHub
```

3. Requisitos

Componente	Versión	Notas
Windows	10 u 11 (64 bits)	`instalador\INSTALAR_TODO.bat` instala todo lo demás. macOS/Linux con `instalador/instalar.sh`
Base de datos	MySQL 8.4 LTS (lo instala `instalador\INSTALAR_TODO.bat`)	Respaldo automático: SQLite incluido en Python
Python	3.10 o superior	Lo instala `instalador\INSTALAR_TODO.bat` si falta
Navegador	Chrome, Edge o Firefox actuales	

4. Instalación

Opción A — Instalación completa en Windows (recomendada)

Funciona en cualquier PC con Windows 10 u 11 de 64 bits, aunque no tenga nada instalado.

1. Descomprima la carpeta del sistema en cualquier lugar.
2. Doble clic en **`instalador\INSTALAR_TODO.bat`** y acepte el permiso de administrador.
3. El instalador hace todo solo (requiere internet durante la instalación):

Paso	Qué hace
1-2	Verifica permisos y copia el sistema a `C:\SistemaCostos`
3	Instala **Python 3.12** si el equipo no tiene Python 3.10+
4	Crea el entorno de Python e instala las librerías
5	Instala **Microsoft Visual C++** si falta (MySQL lo necesita)
6	Descarga **MySQL Server 8.4 LTS** en `C:\SistemaCostos\mysql` (instancia propia, no interfiere con otro MySQL)
7	Inicializa MySQL y lo registra como servicio de Windows **SistemaCostosMySQL** (inicio automático, solo escucha en 127.0.0.1, puerto 3307 o el siguiente libre)
8	Crea la base `sistema_costos`, el usuario `costos` con contraseña aleatoria, asigna una contraseña aleatoria a `root` (queda en `mysql\CLAVE_ROOT_MYSQL.txt`, solo visible para administradores) y actualiza `config.ini`
9	Crea las tablas y los accesos directos en el escritorio y el menú Inicio
10	Abre el sistema en el navegador

4. Ingrese con `admin` / `Admin123*` y **cambie la contraseña**.

Si MySQL no se puede instalar (sin internet, bloqueo de antivirus o políticas del equipo), el instalador **continúa automáticamente con SQLite**: el sistema funciona igual, con la base en `database\sistema_costos.db`. El resumen final indica qué motor quedó en uso.

Instalación sin internet: copie en la carpeta `instalador` el archivo `mysql-8.4.x-winx64.zip` (*Windows (x86, 64-bit), ZIP Archive* de dev.mysql.com), el instalador `python-3.12.x-amd64.exe` y `vc_redist.x64.exe`; el instalador los usa en lugar de descargarlos. El paso de librerías de Python sí necesita internet o una carpeta `.venv` ya preparada.

Uso diario: acceso directo **Sistema de Costos**. La ventana negra que se abre es el servidor: debe permanecer abierta mientras use el sistema.

Desinstalar: menú Inicio › Sistema de Costos › **Desinstalar** (detiene y elimina el servicio de MySQL; pregunta si borrar también los datos).

Registro de la instalación: `%TEMP%\instalacion_sistema_costos.log`.

Opción B — macOS o Linux

```
bash instalador/instalar.sh     # verifica Python 3.10+, instala librerías y crea la base de datos
(SQLite)
./instalador/iniciar.sh         # inicia el sistema y abre el navegador
```

Si el equipo tiene MySQL, puede usarlo con `motor = mysql` en `config.ini` (ver 4.3).

Opción C — Desde código fuente (desarrollo)

```
python -m venv .venv
.venv\Scripts\activate            # Linux/macOS: source .venv/bin/activate
pip install -r backend/requirements.txt
python backend/app.py --init-bd   # crea base de datos y tablas
python backend/app.py             # inicia el sistema
```

4.3 Configurar MySQL manualmente

Solo si no usa `instalador\INSTALAR_TODO.bat` (que ya lo hace) o quiere usar un MySQL existente. Instale MySQL Server 8, cree el usuario y en `config.ini` ponga `motor = mysql` con `usuario`, `contrasena` y `nombre_bd`. La base y las tablas se crean al iniciar. Los datos existentes en SQLite **no** se trasladan automáticamente.

```
CREATE USER 'costos'@'localhost' IDENTIFIED BY 'UnaClaveSegura2026';
GRANT ALL PRIVILEGES ON sistema_costos.* TO 'costos'@'localhost';
FLUSH PRIVILEGES;
```

Luego en `config.ini`: `usuario = costos` y `contrasena = UnaClaveSegura2026`. Si el usuario no tiene permiso para crear bases de datos, cree la base primero: `CREATE DATABASE sistema_costos CHARACTER SET utf8mb4;` o ejecute `mysql -u root -p < database/schema.sql` después de crearla y seleccionarla.

5. Generar el ejecutable (EXE) y el instalador

En Windows, después de `instalar.bat`:

```
instalador\construir_exe.bat
```

- Genera `dist\SistemaCostos\SistemaCostos.exe` (PyInstaller, modo carpeta) con el frontend y el esquema SQL empaquetados. `config\`, `logs\` y `reportes_generados\` quedan junto al EXE para poder editarse.
- Si **Inno Setup 6** está instalado, genera además `instalador\Output\Instalar_SistemaCostos.exe`.

Publicación automática en GitHub: el flujo `.github/workflows/release.yml` ejecuta las pruebas y, en Windows, construye el EXE y el instalador al subir una etiqueta:

```
git tag v1.0.0
git push origin v1.0.0
```

Los archivos `Instalar_SistemaCostos.exe` y `SistemaCostos_portable.zip` quedan adjuntos en la página *Releases* del repositorio.

6. Archivo de configuración (`config/config.ini`)

Sección	Parámetro	Descripción	Por defecto
servidor	host / puerto	Dirección local del sistema	127.0.0.1 / 5000
servidor	abrir_navegador	Abre el navegador al iniciar	si
servidor	depuracion	Modo desarrollo de Flask	no
base_datos	motor	`sqlite` (archivo local) o `mysql` (servidor opcional)	sqlite
base_datos	archivo_sqlite	Ruta del archivo de base de datos	database/sistema_costos.db
base_datos	host, puerto, usuario, contrasena, nombre_bd	Conexión (solo con `motor = mysql`)	127.0.0.1, 3306, root, (vacía), sistema_costos
base_datos	crear_automaticamente	Crea base y tablas si no existen	si
seguridad	duracion_sesion_min	Minutos de vigencia de la sesión	480
seguridad	intentos_maximos / minutos_bloqueo	Bloqueo tras intentos fallidos	5 / 15
seguridad	longitud_minima_contrasena	Longitud mínima de contraseña	8
seguridad	permitir_registro	Muestra «Crear cuenta» en el inicio	si
seguridad	rol_registro	Rol de las cuentas creadas por registro	usuario
administrador_inicial	usuario, contrasena, nombre, correo	Se crea solo si no existe ningún usuario	admin / Admin123*
sistema	nombre, moneda, decimales	Presentación	Sistema de Costos, COP, 2
sistema	horas_laborales_mes	Valor sugerido al crear trabajadores	192
sistema	carpeta_reportes / carpeta_logs	Carpetas de salida	reportes_generados / logs

Los cambios se aplican al reiniciar el sistema.

7. Modelo de datos

Grupo	Tablas
Seguridad	`roles`, `rol_permisos`, `usuarios`, `sesiones`, `bitacora`
Datos generales	`empresas`, `periodos`, `unidades_medida`, `centros_costo`, `proveedores`, `productos`
Materiales	`materias_primas`, `estandar_materiales`
Mano de obra	`cargos`, `prestaciones`, `trabajadores`, `estandar_mano_obra`
CIF	`cif_parametros`, `cif`
Producción	`ordenes_produccion`, `orden_materiales`, `orden_mano_obra`
Resultados	`costos_estandar`, `costos_reales`, `variaciones`

Toda la información de negocio está asociada a `empresa_id`. Las tablas de resultados se recalculan al guardar órdenes, al consultar variaciones y con el botón *Recalcular*.

El sistema se instala **sin datos de negocio**: solo crea los tres roles con sus permisos y el usuario administrador inicial.

8. Fórmulas implementadas (`backend/calculos.py`)

Convención: valor **positivo = desfavorable**, **negativo = favorable**.

- Unidades equivalentes: `UE = terminadas + en proceso × % avance` (materiales y conversión por separado).
- Precio de materiales: `(PR − PE) × QR`
- Cantidad de materiales: `(QR − QE) × PE`, con `QE = cantidad estándar por unidad × UE materiales`
- Tarifa de MOD: `(TR − TE) × HR`
- Eficiencia de MOD: `(HR − HE) × TE`, con `HE = horas estándar por unidad × UE conversión`
- Tarifa real por hora del trabajador: `(salario + auxilio) × (1 + %prestaciones sociales) + salario × %seguridad social y parafiscales`, dividido por horas al mes.
- Tasas de CIF: `tasa fija = CIF fijo presupuestado / nivel normal`; `tasa variable = CIF variable presupuestado / nivel normal`.
- CIF aplicado: `(tasa fija + tasa variable) × base estándar permitida`.
- Presupuesto flexible: `CIF fijo presupuestado + tasa variable × base estándar permitida`.
- Variación de presupuesto: `CIF real − presupuesto flexible`; variación de volumen: `presupuesto flexible − CIF aplicado`.
- CIF real por orden: `CIF real del periodo × base real de la orden / base real del periodo`.

9. API REST (resumen)

Todas las rutas (salvo registro, inicio de sesión y configuración pública) requieren `Authorization: Bearer <token>`. Las rutas por empresa usan la cabecera `X-Empresa: <id>`.

Método	Ruta	Descripción
POST	`/api/auth/registro`	Crear cuenta
POST	`/api/auth/login` · `/api/auth/logout`	Iniciar / cerrar sesión
GET	`/api/auth/yo`	Usuario actual y permisos
POST	`/api/auth/cambiar-contrasena`	Cambio de contraseña
GET/POST/PUT/DELETE	`/api/usuarios[/<id>]`	Administración de usuarios
GET/PUT	`/api/permisos`	Matriz de permisos por rol
GET	`/api/bitacora`	Registro de actividades
GET	`/api/meta`	Definición de catálogos (campos y reglas)
GET/POST/PUT/DELETE	`/api/r/<recurso>[/<id>]`	Catálogos: empresas, periodos, unidades_medida, centros_costo, proveedores, productos, materias_primas, estandar_materiales, cargos, prestaciones, trabajadores, estandar_mano_obra, cif_parametros, cif
GET/POST/PUT/DELETE	`/api/ordenes[/<id>]`	Órdenes de producción con sus consumos
GET	`/api/ordenes/<id>/hoja`	Hoja de costos de la orden
GET	`/api/estandar/<producto_id>?periodo_id=`	Hoja de costo estándar unitaria
GET	`/api/variaciones?periodo_id=&producto_id=`	Análisis de variaciones
POST	`/api/recalcular`	Recalcula costos y variaciones del periodo
GET	`/api/dashboard?periodo_id=&producto_id=`	Indicadores
GET	`/api/reportes/<tipo>?formato=json|xlsx|pdf`	Reportes
GET	`/api/salud`	Estado del servidor y la base de datos

Errores: `400` validación (`{error, errores: {campo: mensaje}}`), `401` sesión, `403` permiso, `404` , `409` integridad, `503` base de datos.

10. Seguridad

- Contraseñas con hash PBKDF2-SHA256 (600 000 iteraciones); nunca se almacenan en texto plano.
- Token de sesión aleatorio con vencimiento; se invalida al cerrar sesión o desactivar el usuario.
- Bloqueo temporal por intentos fallidos.
- Permisos por rol y módulo (ver, crear, editar, eliminar) verificados en el servidor.
- Registro de actividades: inicios/cierres de sesión, intentos fallidos, creación de cuentas, altas, cambios, eliminaciones, exportaciones.
- Consultas parametrizadas (sin concatenar datos del usuario en SQL).
- El servidor escucha solo en `127.0.0.1`. Para uso en red local cambie `host = 0.0.0.0` bajo su responsabilidad.

11. Pruebas

```
python -m unittest discover -s tests -v
```

Usan una base SQLite temporal (no tocan la base real) y verifican un caso de costeo con resultados conocidos, registro e inicio de sesión, permisos, validaciones, integridad, cierre de periodo y la generación de todos los reportes en Excel y PDF.

12. Copias de seguridad

En el sistema, *Usuarios y seguridad › Copia de seguridad › Descargar* (administrador) genera una copia consistente aunque haya usuarios trabajando: un archivo **.sql** con MySQL (usa el `mysqldump` instalado) o un archivo **.db** con SQLite.

SQLite: También puede copiar `database\sistema_costos.db` con el sistema cerrado.

Restaurar: cierre el sistema, renombre el archivo actual y ponga la copia como `database\sistema_costos.db`.

MySQL (opcional):

```
"C:\Program Files\MySQL\MySQL Server 8.0\bin\mysqldump" -u root -p sistema_costos > respaldo_costos.sql
:: Restaurar
"C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql" -u root -p sistema_costos < respaldo_costos.sql
```

13. Solución de problemas

Síntoma	Causa probable	Solución
«database is locked»	Otro programa tiene abierto el archivo .db	Cierre visores de SQLite u otras copias del sistema
«unable to open database file»	La carpeta `database` no permite escritura	Dé permiso de modificación a la carpeta o instale en `C:\SistemaCostos`
(MySQL) «No fue posible conectar con MySQL»	Servicio detenido o credenciales erróneas	Inicie el servicio *MySQL80* y revise `config.ini`
El navegador no abre	`abrir_navegador = no` o bloqueo	Abra `http://127.0.0.1:5000` manualmente
«Address already in use»	Otro programa usa el puerto 5000	Cambie `puerto` en `[servidor]`
Olvidó la contraseña de admin	—	Otro administrador puede asignar una nueva en *Usuarios*. Si no hay otro, cierre el sistema y ejecute `SistemaCostos.exe --restablecer-admin` (o `python backend/app.py --restablecer-admin`): la contraseña vuelve al valor de `[administrador_inicial]` en `config.ini`
Error inesperado	—	Revise `logs/sistema.log`